

Integración React-Redux: conectando la UI con el estado global

En este documento, se explicará cómo integrar Redux con React, conectar el store al árbol de componentes y manipular el estado global mediante hooks.

Objetivo general

Integrar *Redux Toolkit* en un proyecto *React-Vite* para administrar estados globales de manera eficiente y estructurada.

Subtemas:

1. Instalación de dependencias.
2. Configuración del store global.
3. Uso de `<Provider>`.
4. Hooks de React-Redux (`useSelector`, `useDispatch`).
5. Flujo completo: acción → reducer → componente.

Instalación de dependencias

```
npm install @reduxjs/toolkit react-redux axios
```

- `@reduxjs/toolkit`: herramientas para gestionar el estado global.
- `react-redux`: conecta *Redux* con *React*.
- *Axios*: cliente HTTP para consumir APIs REST (más flexible que `fetch`).

Axios simplifica la gestión de headers, tokens, interceptores y manejo de errores.

Configuración del Store

```
import { configureStore } from '@reduxjs/toolkit';
import userReducer from '../features/user/userSlice';

export const store = configureStore({
  reducer: {
    user: userReducer
  }
});
```

- Se centralizan todos los reducers.
- *Redux Toolkit* aplica automáticamente middleware (como `Thunk`).

Tip: Cada módulo de tu aplicación tiene que tener su propio *slice* (uno por entidad en la DB).

Conexión del Store con el Proyecto

Archivo: /src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import { Provider } from 'react-redux';
import { store } from './app/store.js';

ReactDOM.createRoot(document.getElementById('root')).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

El componente `<Provider>` permite que todos los componentes hijos accedan al *store* global sin necesidad de pasar props manualmente.

Acceso al estado global

Hook: `useSelector()`

Toma una función selectora como argumento para extraer la información deseada, devolviendo el valor resultante. El componente se volverá a renderizar si el valor devuelto por el selector cambia.

```
import { useSelector } from 'react-redux';

function UserProfile() {
  const user = useSelector((state) => state.user);
  return <h2>Hola, {user.name}</h2>;
}
```

Funcionalidad principal:

- **Extraer datos:** Se usa en componentes de función para obtener datos del store de Redux.
- **Suscripción al estado:** Se suscribe al *Store Redux* y se ejecuta cada vez que se produce una acción, para luego ejecutar la función selectora y comprobar si el estado ha cambiado.
- **Optimización automática:** Si el valor devuelto por el selector no cambia, el componente no se volverá a renderizar.

Hook: useDispatch()

Permite a los componentes de *React* enviar (despachar) acciones para modificar el estado de la aplicación. Al llamar a este *hook*, se obtiene una referencia a la función *dispatch* de *Redux*, la cual se utiliza para pasar acciones que serán procesadas por los *reducers*.

```
import { useDispatch } from 'react-redux';
import { login } from '../features/user/userSlice';

function LoginButton() {
  const dispatch = useDispatch();
  return (
    <button onClick={() => dispatch(login('Juan'))}>
      Iniciar sesión
    </button>
  );
}
```

- **Función principal:** Proporciona acceso directo a la función *dispatch* de la tienda *Redux*, haciendo que sea más fácil para los componentes funcionales activar cambios en el estado global.

Integración con Axios

Peticiones HTTP con Axios y Redux Thunk

```
// userThunks.js
import axios from 'axios';
import { setUsers, setError } from './usersSlice';

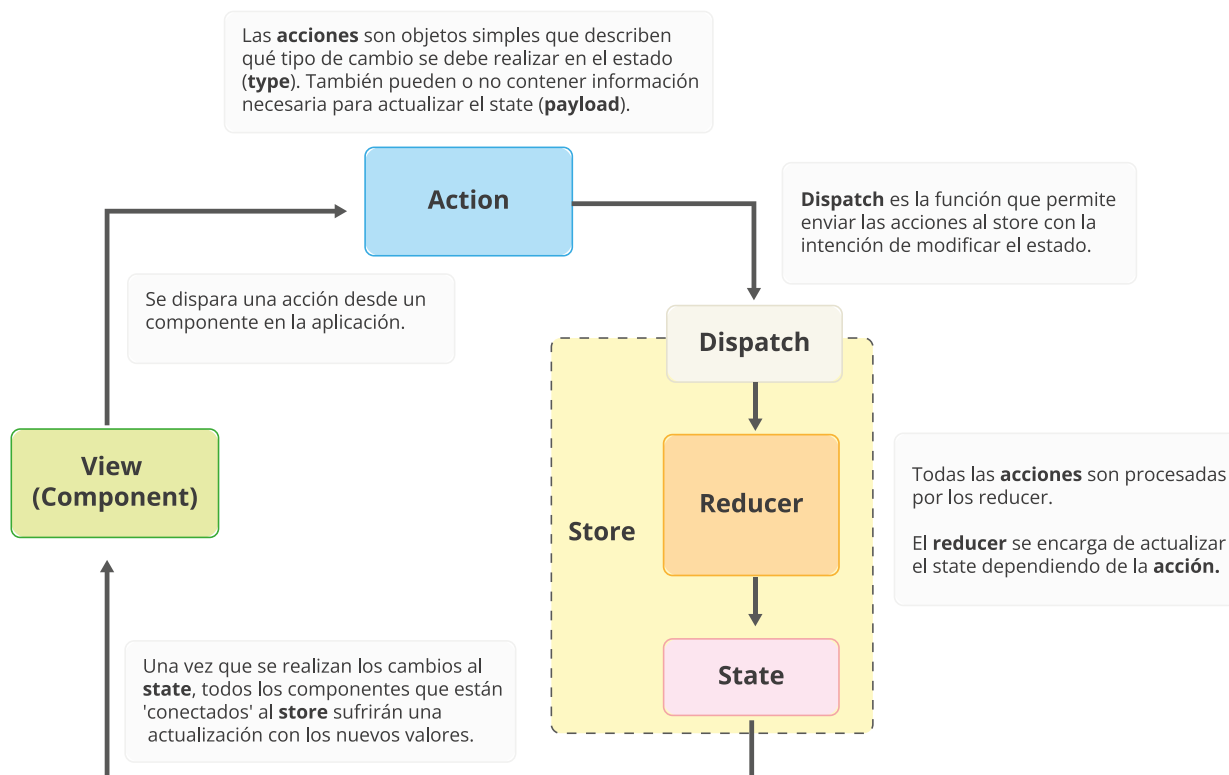
export const fetchUsers = () => async (dispatch) => {
  try {
    const { data } = await axios.get('https://jsonplaceholder.typicode.com/users');
    dispatch(setUsers(data));
  } catch (error) {
    dispatch(setError(error.message));
  }
};
```

- *axios.get()* realiza la llamada HTTP.
- Si responde correctamente, se actualiza el store con *dispatch(setUsers(data))*.
- Si ocurre un error, se captura en *setError*.

Axios permite simplificar el flujo asíncrono y mantener el código más limpio y estructurado.

Flujo completo con Axios + Redux

- Componente ejecuta `dispatch(fetchUsers())`.
- El *thunk* usa *Axios* para traer los datos del backend.
- El reducer actualiza el estado global.
- Los componentes que usan `useSelector` se actualizan automáticamente.



Bibliografía

- REDUX TOOLKIT. En Redux Toolkit Docs [en línea]. [consulta: 3 de diciembre de 2025]. Disponible en: <https://redux-toolkit.js.org>
- REACT REDUX. En React Redux Docs [en línea]. [consulta: 3 de diciembre de 2025]. Disponible en: <https://react-redux.js.org/>
- AXIOS. En Axios Docs [en línea]. [consulta: 3 de diciembre de 2025]. Disponible en: <https://axios-http.com/docs/intro>